AI & Software Engineering Workshop

Week 1, Day 1: Setup and First Message

Edik Simonian, Summer 2026

First, let's meet the room

Go around, **30 seconds each**:

- **Who are you?** Your name and grade
- **What have you made or messed with?** Any project, game, app, website, mod, art, anything you've built or tinkered with
- **What do you want to build?** Something you look forward to creating or programming

No wrong answers, even "nothing yet" is fine. This is where we find out what to build by Friday.

See it working first

t.me/tele_pythonanywhere_bot

This is where you'll be by **Friday**: your own bot, your personality, live on the internet.

Go ahead, message it now, then we'll build one from scratch.

What you'll build this week

- A **Telegram bot** powered by a real LLM
- With its own **personality**, you design it
- Custom **slash commands**, you code them
- **Memory** that survives restarts
- **Live on the internet** by Friday, running even when your computer is off

By the end of today: the bot replies to you, from code on your machine.

The week, day by day

- **Day 1, Setup and First Message** *(today)*: fork the repo, get your tokens, `make run`, and ship your first `/start` edit.
- **Day 2, Personality and Prompts**: the system prompt, the one line that gives your bot a voice and a persona.
- **Day 3, New Commands**: live-code `/joke`, then build your own like `/quote` and `/roast`, with a test.
- **Day 4, Memory and Deploy**: SQLite memory that survives restarts, then going live on PythonAnywhere by webhook.
- **Day 5, Build Your Own Bot**: design and ship a feature that is all yours, then demo to the room.

Each day builds on the last. By Friday it is yours, and it is online.

What is a bot?

Two ways Telegram talks to your code:

- **Polling**: your code keeps asking Telegram *"any new messages?"*
  → what we use **today**, on your computer
- **Webhook**: Telegram pushes each message to your server's URL
  → what we use **Day 4**, in production

Same bot code either way. Only the delivery changes.

The stack

Telegram → Flask (Python) → Cerebras → reply

- **Telegram Bot API**: the messaging interface
- **Flask**: receives the messages
- **Cerebras**: runs the LLM that writes the replies *(free tier)*
- **SQLite**: memory *(Day 4)*
- **PythonAnywhere**: hosting *(Day 4)*
- **GitHub**: your code, your tests, your deploys

Setup 1 of 3: GitHub

1. Create a GitHub account *(if you don't have one)*
2. **Fork** the repo: your own copy, needed for auto-deploy on Day 4
3. Clone your fork and install:

```
navigate https://github.com/EdikSimonian/telegram-pythonanywhere-bot.git
git clone https://github.com/<your-username>/telegram-pythonanywhere-bot.git
cd telegram-pythonanywhere-bot
make install
```

Setup 2 of 3: Telegram bot token

1. In Telegram, search for **@BotFather**
2. Send `/newbot`
3. Pick a name, then a username ending in `bot`
4. Copy the **token** (looks like `7123456789:AAF...`)

Setup 3 of 3: AI API key

1. Sign up at **cloud.cerebras.ai** *(free, no credit card)*
2. Profile icon → **API Keys** → **Create new API key**
3. Copy the key (looks like `csk-...`)

Wire it up: `.env`

```
cp .env.example .env
```

Set two lines:

```
TELEGRAM_BOT_TOKEN=<your BotFather token>
AI_API_KEY=<your Cerebras key>
```

Leave the rest as-is.

> *Secrets live in `.env`. Never in code, never in git.*

First contact

```
make run
```

```
Bot @your_bot_username starting in polling mode.
Send your bot a message on Telegram to try it out.
```

Message your bot. Watch the exchange appear in your terminal.

"Stateless mode" is expected. Memory arrives on Day 4.

Prove it works, like an engineer

```
make test
```

- The whole suite runs **offline**: no API keys, no network
- The **same suite** runs in GitHub Actions on every push to your fork
- Green check on GitHub = your bot's logic still works

Your first code edit

`bot/handlers.py`:

```
@bot.message_handler(commands=["start"], func=is_allowed)
def cmd_start(message):
    bot.send_message(
        message.chat.id,
        "Hello! I'm your AI assistant...",
    )
```

Change the greeting → `Ctrl+C` → `make run` → send `/start`.

You just shipped your first change.

Today → Tomorrow

Today the bot works on your computer and answers as a generic assistant.

Tomorrow: the most powerful single line in the project, the **system prompt**. Your bot gets a personality.